Blatt 04: Stream-API & Damaged Bridge (Git Remote, Streams, Lambda-Ausdrücke)

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Hinweis: Bitte denken Sie daran, dass Sie spätestens ab diesem Blatt die Bearbeitung der Aufgaben in einem öffentlich sichtbaren Git-Repo durchführen sollen. Den Link zu Ihrem jeweiligen Lösungs-Repo geben Sie bitte in Ihrem *Post Mortem* mit an.

Stream-API

Git: Pull-Requests (und Code-Formatierung und -Dokumentation)

Forken Sie das "Stream-API"-Repo und erzeugen Sie eine lokale Arbeitskopie von Ihrem Fork.

Sie finden die Vorgaben für jede Teilaufgabe in einem eigenen Branch. Checken Sie diese Branches lokal als Tracking-Branches aus und bearbeiten Sie die Teilaufgaben jeweils in ihrem eigenen Branch. Pushen Sie Ihre Änderungen in Ihren Fork zurück und erstellen Sie dort je einen Pull-Request auf **Ihren** eigenen `master`-Branch.

Bitte lassen Sie die Pull-Requests bis zur Vorstellung im Praktikum offen.

Achten Sie darauf, alle Schritte nachvollziehbar in Ihrer Arbeitskopie per Git-Commit festzuhalten. Demonstrieren Sie im Praktikum, wie Sie mit den Pull-Requests arbeiten.

Stream-API: Task I

Betrachten Sie den Branch `task_i`. Sie finden im Package `streamapi` einige Hilfsklassen sowie in der Datei `Main.java` einen Starter für diese erste Teilaufgabe.

In der Funktion `Main#students` wird für eine Liste von `Student`-Objekten die Summe der gesammelten ECTS berechnet.

Schreiben Sie den Body dieser Methode so um, dass die selbe Funktionalität unter Nutzung der

DevDungeon: Zerbrechende Tiles und Speed Potions (Lambda-Ausdrücke)

Klonen Sie das Projekt DevDungeon und laden Sie es in Ihrer IDE als Gradle-Projekt. Betrachten Sie das Sub-Projekt[1] "devDungeon". Dies ist ein von einem Studierenden (@Flamtky) erstelltes Spiel mit mehreren Leveln, in denen Sie spielerisch verschiedene Aufgaben *in-game* und *ex-game* lösen müssen.

Starten Sie den DevDungeon mit `./gradlew devDungeon:runDevDungeon`. Spielen Sie sich für diese Aufgabe durch das **erste Level** ("Damaged Bridge")[2].

Ziel ist es, die mysteriöse Brücke in der Mitte des ersten Levels lebendig zu überqueren. Beobachten Sie die Startsequenz: Was fällt ihnen an dem Monster auf, dass Sie ganz am Anfang angreifen will? Sie finden weitere Hinweise in den Briefkästen und über die Popups ... Beachten Sie auch die Hinweise am versteckten Item. Um dieses nutzbar zu machen, müssen Sie in den Java-Code des Spiels gehen (im `src/`-Unterordner im Sub-Projekt "devDungeon") und den Effekt für das Item reparieren (implementieren). Analysieren Sie den Code für das Item und seinen Effekt, und schauen Sie sich die anderen Effekte im selben Package an. Schreiben Sie nun Code für die mit "TODO" markierte Methode des Effekts. Starten Sie dann das Spiel neu und schauen Sie, ob das Item nun funktioniert.

WICHTIG: **Bevor** Sie mit der Implementierung beginnen, schauen Sie sich bitte die Einführung in die Programmierung des Dungeons und die verwendete *Entity-Component-System*-Architektur in der Lektion Intro Dungeon an. Dort werden Ihnen Hintergründe zum Dungeon und der für diese Aufgabe relevanten Component erklärt.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Last modified: *c1867fd (tooling: remove numbering of homework tasks, 2025-04-23)*